# Hash functions are a critical component of information security, primarily used to ensure **data integrity** and securely store passwords.[1] They are mathematical algorithms that take an input (of any size) and convert it into a fixed-size, unique string of characters called a **hash value** or **message digest**.[2]

The key idea is that the process is **one-way** (irreversible).[3]

---

## 1. What is a Hash Function? 🔐

A hash function is a formula that performs a one-way transformation on data.[4]

- **Input (Message):** Any piece of data, such as a file, a password, or a sentence (e.g., "The cat sat on the mat.").[5]
- **Hash Function (Algorithm):** A fixed mathematical process (e.g., [6]$\text{SHA-256}$).[7]
- **Output (Hash Value/Digest):** A fixed-length string of characters (e.g., [8]$\text{a1d0f...7b3c}$).[9]

**Crucial Analogy:** A hash is like a digital **fingerprint** of the data.[10] No matter how big the input is (a single word or an entire movie), the fingerprint is always the same length.[11]

---

## 2. Key Properties of Secure Hash Functions

For a hash function to be useful in cryptography, it must have specific properties:

| Property | Explanation |
| --- | --- |

| | |
|---|---|
| **Deterministic** | The same input will *always* produce the exact same hash output. |
| **Fixed Output Size** | The output (hash) is always the same length, regardless of the input size. |
| **One-Way (Irreversible)** | It's computationally impossible (or infeasible) to reconstruct the original input from the hash value. **You can't "de-hash" it.** |
| **Pre-Image Resistance** | Given a hash ($h$), it's extremely hard to find any input ($m$) that produces that hash. (Supports the one-way nature). |
| **Avalanche Effect** | A tiny change in the input (like changing one comma) results in a drastically different, unpredictable hash output. |
| **Collision Resistance** | It should be extremely difficult to find two different inputs ($m_1$ and $m_2$) that produce the exact same hash value ($h$). |

---

## 3. Notes in Q&A Format (Easy Examples)

**Q1: What is the main security purpose of a hash function?**

**Answer:** To ensure **data integrity**.[12]

- **Example:** When you download a software file, the website often provides the file's $\text{SHA-256}$ hash.
  1. The provider hashes the original file: **File $\rightarrow$ Hash ($\text{ABC123...}$)**
  2. You download the file and run the same hash function on it: **Your Downloaded File $\rightarrow$ Hash ($\text{???}$)**

3. If your calculated hash ([13]$\text{???}$) **matches** the published hash ([14]$\text{ABC123...}$), you know the file hasn't been corrupted or tampered with by a malicious third party during the download.[15]

### Q2: How are hash functions used to secure user passwords?

**Answer:** By storing the **hash** of the password, not the password itself (plaintext).[16]

- **Scenario:** A user signs up with the password "**mysecret**".
  1. The system calculates: $\text{Hash}(\text{"mysecret"}) \rightarrow \text{Stored Hash ($\text{X Y Z}$)}$
  2. The system stores only $\text{X Y Z}$ in the database.
  3. If the database is hacked, the attacker only gets [17]$\text{X Y Z}$, not the original password.[18] Because hashing is **one-way**, they cannot easily recover the original "**mysecret**".[19]
- **Login Process:** When the user attempts to log in, the system hashes the entered password and checks if the new hash matches the stored hash [20]$\text{X Y Z}$.[21]

### Q3: What is the "Avalanche Effect" and why is it important for security?

**Answer:** The **Avalanche Effect** means a minimal change in the input causes a complete change in the output hash.[22] This is essential for preventing attackers from guessing similar passwords.[23]

- **Example:**
  - $\text{Input 1}$: **Hello**
  - $\text{SHA-256 Hash}$: $\text{185F8DB32271FE254F06A58498007CC196777B670F570B34507914E486C2750A}$
  - $\text{Input 2}$: **Jello** (One letter changed)
  - $\text{SHA-256 Hash}$: $\text{80C2CC0C33887EC922B2F8D58045F19602E9B4F817B27C2BEE00D28E12E99

B96}$

- As you can see, the hashes are completely different, making it impossible to infer the original input from small changes to the hash.[24]

**Q4: Name some popular hashing algorithms.**

**Answer:**

- **$\text{SHA-256}$** (Secure Hash Algorithm with a 256-bit output): Widely considered the current standard for security, used in **Bitcoin** and many modern security applications.[25]

- **$\text{SHA-1}$** (160-bit output): Now considered insecure and deprecated for most uses due to known collision vulnerabilities.[26]

- **$\text{MD5}$** (Message Digest 5, 128-bit output): Highly insecure due to easily found collisions; should **never** be used for password storage or digital signatures.